

Raft 一致性算法实现

邓枭

(MF2133017 南京大学 计算机科学与技术系, 南京 210023, E-mail: dengxiao@smail.nju.edu.cn)

1 选举 (Part I)

从面向对象或面向过程的角度看, 选举至少有这么几个过程: 发送选举、选民决定给谁投票、统计投票结果、通知大家谁赢得了选举。

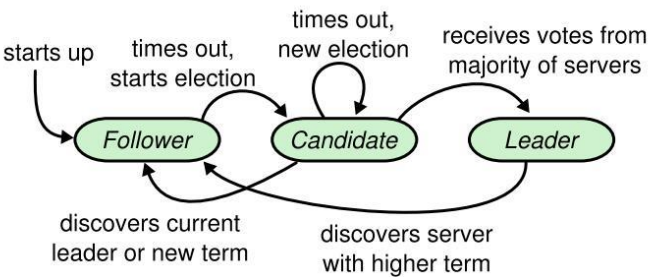


图 1 一个 server 的状态转移过程

```
type Raft struct {
    mu          sync.Mutex
    peers       []*labrpc.ClientEnd
    persister   *Persister
    me          int // index into peers[]
    // Your data here.
    currentTerm int // latest term server has seen (initialized to 0 on first
                    // boot, increases monotonically)
    votedFor    int // candidateId that received vote in current term (or null
                    // if none)
    logs        []LogEntry //each entry contains command for state machine, and term
                    // when entry was received by leader
    // Volatile data
    state       int // 0 leader 1 candidate 2 follower
    votesCount  int // how many votes I got
    timer       *time.Timer
}
```

如图 1 所示, 系统刚启动时, 每个 server 都是 follower, 并且每个 server 都设置了定时器。如果一个 server 在定时器计时的这段时间内没有收到 leader 的任何消息, 则认为 leader 已经出现故障, 遂发起一轮新选举, 并推荐它自己为 leader, 即它进入 candidate 状态。然后调用其他 server 的 [RequestVote](#) RPC, 即告诉它们

```
type RequestVoteArgs struct {
    Term          int // candidate's term
    CandidateId    int // who requesting vote
    LastLogIndex  int // index of candidate's last log entry (§5.4)
```

```

    LastLogTerm    int           // term of candidate's last log entry (§5.4)
}

```

自己的任期号：Term、自己是谁：CandidateId、自己 logs 中最后一个 entry 的下标：LastLogIndex、自己 logs 中最后一个 entry 是由哪个任期的 leader 复制过来的：LastLogTerm。

一个 server 如果收到一个投票邀请 **RequestVote**，则决定是否给它投票。怎么投票的断决过程请参考 Raft 论文或我的代码。

```

type RequestVoteReply struct {
    // Your data here.
    Term      int // currentTerm, for candidate to update itself
    VoteGranted bool // true means candidate received vote
}

```

candidate 用 **getVoteResult** 获取投票结果，如果得票超过半数，则它成为新的 leader。

如果只是为了通过“Election”这个测试样例，只需要实现：

- **Timeout/ResetTimer**：重置、定时触发 leader 的 heartbeat，或可能会发生的选举。
- **RequestVote**：是个远程调用（RPC），candidate 调用其他 server 的 **RequestVote** 来发起一次投票。
- **getVoteResult**：每个成功的 **RequestVote** 都会触发 **getVoteResult** 来统计投票结果。
- **GetState**：返回当前 server 是否是 leader，和该 server 的 Term。
- **Make**：初始化一个 server，记得要重置定时器，否则没有 server 会发起选举。
- **AppendEntries**：在目前只是为了发送 heartbeat，否则会出现如图 2 的瑕疵。

从下图 2 中的 warning 可以想到，如果不实现 **AppendEntries** 虽然已经能 pass “Election” 测试，不过因为没有发送 heartbeat 的机制，所以总会出现新的选举，即每个 leader 只能任一期，系统的最大 Term 一直在增大，虽然系统没有出现多个 leader，也不是没有 leader，但每个 leader 只能任一期总是不好的。

```

PS C:\Users\dengxiao\go\src\raft> go test -run Election
Test: initial election ...
warning: term changed even though there were no failures ... Passed
Test: election after network failure ...
... Passed
PASS
ok      raft    7.205s
PS C:\Users\dengxiao\go\src\raft>

```

图 2 没有 heartbeat 时的结果

所以，需要实现 **AppendEntries** 来完成 heartbeat 发送，这时的 **AppendEntryArgs** 还用不到 **Entry**、**LeaderCommit** 等一些 Fig.2 中标有的参数。如图 3 所示，用实现 **AppendEntries** 实现 heartbeat 机制后就可以完美通过“Election”测试了。

```

type AppendEntryArgs struct {
    Term      int // leader's term
    LeaderId   int
    PrevLogIndex int // index of log entry immediately preceding new ones
    PrevLogTerm int // term of prevLogIndex entry
}

```

```

type AppendEntryReply struct {
    Term      int
    Success    bool
}

```

```

PS C:\Users\dengxiao\go\src\raft> go test -run Election
Test: initial election ...
... Passed
Test: election after network failure ...
... Passed
PASS
ok      raft      7.227s
PS C:\Users\dengxiao\go\src\raft>

```

图 3 用 AppendEntries 实现 heartbeat 机制后的结果

2 Entry 的增加和复制 (Part II)

到了这一部分，功能已经有点复杂，为了代码可读性，把调用所有其他 server 的 `RequestVote` 的循环结构提取出来单独作为 `SendRequestVoteToAll` 函数。同样地，把调用所有其他 server 的 `AppendEntries` 的循环结构提取出来作为 `SendAppendEntriesToAll` 函数。由于代码增多，并且本节代码也没有被最终代码覆盖，所以本小节不贴代码，需要看代码请参阅附件中的代码。

raft.go 中有如下函数：

1. `RequestVote`：是一个 RPC (Remote Procedure Call)，用于其他 server 决定是否给 candidate 投票。
 2. `SendRequestVoteToAll`：用于让 candidate 发起一次选举，即 candidate 调用所有其他 server 的 `RequestVote`。
 3. `getVoteResult`：用于 candidate 统计选举结果，每个 `RequestVote` 调用成功后都会触发 `getVoteResult` 统计投票结果。
 4. `AppendEntries`：是一个 RPC，leader 调用 follower 的 `AppendEntries` 来通知他们自己是 leader，或向 follower 复制 log entry，同时 follower 可以判断有没有新的 command 可以 commit。
 5. `SendAppendEntriesToAll`：leader 在赢得选举后，在 `SendAppendEntriesToAll` 里调用 follower 的 `AppendEntries` 来通知他们自己是新的 leader；或有 client 发来新的 command 时，向 follower 复制 log entry。
 6. `AfterSendAppendEntries`：在每个成功的 `AppendEntries` RPC 调用成功后，都会触发 leader 的 `AfterSendAppendEntries` 来调整该 follower 的 nextIndex 和 matchIndex。
 7. `Timeout`：定时器，每个 server 都有定时器。leader 的定时器定时触发 heartbeat 来通知 follower 自己还没掉线。follower 用于一段时间内没收到 leader 的消息时，认为以前的 leader 已经发生故障，所以需要发起新的选举。
 8. `Commit`：用于 commit 已经被复制到超过半数 server 上的 command。
 9. `ResetTimer`：用于重置各个 server 的定时器。
 10. `Make`：用于启动一个 server。
 11. `Start`：用于让 client 向 leader 发送一个命令
- candidate 使用 `SendRequestVoteToAll` 来调用所有其他 server 的 `RequestVote` RPC，让它们为自己投票。每个成功的 `RequestVote` RPC 都会触发 `getVoteResult` 来统计投票结果。
- leader 使用 `SendAppendEntriesToAll` 来调用 follower 的 `AppendEntries` RPC，来向 follower 复制 entries 或发送普通的 heartbeat。每个成功的 `AppendEntries` RPC 都会触发 `AfterSendAppendEntries` 来调整 follower 的 nextIndex 和 matchIndex。并根据 `AppendEntries` 的结果看看有没有新的 command 可以被 commit，若有就调用 `Commit` 提交。

以下各个函数中的要点是我个人调试过程中总结的经验，全部实现这些要点不等于就能通过测试用例，也就是这些要点是“包括但不限于”。

2.1 RequestVote

函数原型：`func (rf *Raft) RequestVote(args RequestVoteArgs, reply *RequestVoteReply)`

用于决定是否给某个 server 投出一票。

需要注意以下要点：

- 当 candidate 的 `term < rf.currentTerm`（自己掌握的最新的任期号），不能给他投票
- 当 candidate 的 `term == rf.currentTerm`，如果已经给别人投过票，则不能再投这一票
- 如果碰到更大的任期号，必须马上转换为 follower，不管自己当前时 leader 还是 candidate，并更新自己的 `votedFor`（历史投票记录）和存储的任期号
- 判断 candidate 的 logs 是不是比自己的 logs 更新，只有当 candidate 的更新时才能给他投票。谁的最后一个 log 的 term 更大谁更新，如果 term 一样，则 logs 更长的更新。
- 最重要的一点：只有当真正投出一票时才能重置自己的定时器

2.2 SendRequestVoteToAll

函数原型：`func (rf *Raft) SendRequestVoteToAll(args RequestVoteArgs)`

用于向所有其他 server 发出投票请求，让它们给自己投票。

需要注意以下要点：

- 不要调用自己的 `RequestVote`

2.3 getVoteResult

函数原型：`func (rf *Raft) getVoteResult(reply RequestVoteReply)`

用于统计自己发起的选举的结果。

需要注意以下要点：

- 只有当 `reply.Term == rf.currentTerm` 时才能认为此票有效。如果 `reply.Term < rf.currentTerm` 它可能是由于网络延时或投票者故障导致的，是前轮投票的结果，而不是本轮投票的结果。
- 当 `reply.Term > rf.currentTerm` 时，说明遇到了更大的任期号，要立即转为 follower。
- 当 `reply.Term == rf.currentTerm && reply.VoteGranted == true` 时，还要看看自己是不是 candidate，因为可能发起选举期间自己遇到了更大的 term，从而自己已经不是 candidate 了，即自己发起的选举已经结束了，再统计票数已经没有意义了。**但是**，不能在刚进入 `getVoteResult` 函数时发现自己已经不是 candidate 就直接退出函数，因为有可能还会遇到更大的 term，从而还需要更新自己的 `currentTerm`。
- **极为重要**：每次成为新的 leader 都要重置自己的 `nextIndex` 数组和 `matchIndex` 数组，Raft 论文中说 `matchIndex` 应该初始化为“0”，但它的有效下标是从“1”开始，而我的代码中有效下标从“0”开始，所以我应该初始化为“-1”。
- 当获取的票数超过一半时，马上调用 `SendAppendEntriesToAll`，通知其他 server，自己是新的 leader。这时 `append` 的 `Entries` 参数是空的，因为所有 `nextIndex` 被初始化为 leader 的最后一个 entry 的下标+1。
- 调用 `SendAppendEntriesToAll` 之后要马上重置定时器，以保证至少在 `heartbeatTime` 时间间隔之内再次联系过其他 server。

2.4 AppendEntries

函数原型: `func (rf *Raft) AppendEntries(args AppendEntryArgs, reply *AppendEntryReply)`

用于让 leader 向 follower 复制 logs。只有认为自己是 leader 的 server 才能调用其他 server 的 `AppendEntries`。需要注意以下要点:

- 如果 `args.Term < rf.currentTerm`, 说明该 “leader” 现在已经不是 leader, 所以不能听它的。并且咱还好心地告诉它, 现在系统中更大的 Term 至少是 `rf.currentTerm`, 所以你还是老老实实当 follower 吧。发生这种情况可能是因为网络延迟, 或者在不拥有超过半数 server 另一个网络分区的老 leader 刚刚并入正常的网络分区。
- `args.Term >= rf.currentTerm` 时, 如果在 `args.PrevLogIndex` 不包含有 entry 或者该 entry 的 term 和 `args.PrevLogTerm` 不一样, 则回复 leader 说 `AppendEntries` 失败, 并且删除 follower 本地该 entry 及其之后的所有 entry。
- If `leaderCommit > commitIndex`, set `commitIndex = min(leaderCommit, index of last new entry)`
- **重点:** Raft 论文中说如果 `AppendEntries` 失败, 则 “decrement nextIndex and retry”。没错, 是需要 “decrement nextIndex”, 但怎么减小 nextIndex 是有讲究的。如果每次失败都只是 “-1” 那实在太慢了。每次都让 `nextIndex = 0`, 那当然可以, 但又太极端了。我让 `AppendEntries` 给 leader 返回一个值 `MatchIndex`, 告诉 leader 下次 `AppendEntries` 应该从 `MatchIndex+1` 开始。返回的 `MatchIndex` 是这么确定的: 如果本地 logs 中的 Term 有和 `PrevLogTerm` 一样的, 则 `MatchIndex = 该 Term 的下标`, 那么 leader 的 `nextIndex = MatchIndex+1`, 这样能保证下次 “`args.PrevLogTerm == rf.logs[args.PrevLogIndex].Term`”。否则还是让 leader 的 `nextIndex - XLen`。这里的 `XLen` 是一个大于 0 的常数, 我这里取了 10。并且需要保证 `MatchIndex ≥ -1`, 即保证 `nextIndex ≥ 0`。因为每次减 1 太慢了, 会导致 leader 花很多时间在复制 log 上, 从而在其他地方可能出现超时。从直觉上看, 肯定有其他更高效的方法, 待日后研究吧。光是发现问题就花了不少时间。当然此问题也可以通过增大 heartbeat 时间间隔和选举时间解决。

2.5 SendAppendEntriesToAll

函数原型: `func (rf *Raft) SendAppendEntriesToAll()`

用于向所有 follower 发送 entries, entries 可以是空的, 例如发送 heartbeat 时 entries 是空的。只能由认为自己是 leader 的 server 调用。

需要注意以下要点:

- 不要调用自己的 `AppendEntries`
- 注意 `PrevLogIndex` 不能小于 0
- `args.Entries` 包含下标为 `nextIndex` 及其之后的所有 entries, 是一个数组, 不是只有一条 entry。

2.6 AfterSendAppendEntries

函数原型: `func (rf *Raft) AfterSendAppendEntries(peer int, reply AppendEntryReply)`

成功调用 follower 的 `AppendEntries` 并返回后, 会触发 leader 的 `AfterSendAppendEntries` 函数, 用于获取 `AppendEntries` 的结果, 如果失败则调整 `nextIndex`, 然后重新调用 `AppendEntries`。

需要注意以下要点:

- 此函数对非 leader 的 server 没有用, 所有当发现自己已经不是 leader 时, 可以直接退出函数。网络延迟会导致这种情况发生。
- 任何时候发现有 `term > rf.currentTerm`, 都要转换为 follower。
- 如果 `AppendEntries` 失败, 则置 “`rf.nextIndex[peer] = reply.MatchIndex + 1`”, 后重新调用

AppendEntries。其中 `reply.MatchIndex` 的含义如 3.4 节所述。

- 如果 **AppendEntries** 成功，则看是不是超过半数的 server 的 `matchIndex` 都 $\geq$ 该 follower 的 `matchIndex`，如果是，则更新 leader 的 `commitIndex`。

2.7 TimeOut

函数原型：`func (rf *Raft) TimeOut()`

每个 server 的定时器会定时触发 **TimeOut()** 函数，如果是 leader，在发送一次 heartbeat。如果是 follower，定时器被触发说明已经很长时间没有收到 leader 的消息，则认为 leader 已经出现故障，所以发起新的选举。

需要注意以下要点：

- 执行完所需的操作后，记得重置定时器。

2.8 Commit

函数原型：`func (rf *Raft) Commit()`

用于提交一些已经被超过半数 server 复制的 command 到 state machine。

需要注意以下要点：

- **要点**：`args.Index` 要等于 `entry` 的下标+1，因为论文和测试用例的有效下标是从 1 开始，而我的代码的下标从 0 开始

2.9 ResetTimer

函数原型：`func (rf *Raft) ResetTimer()`

用于重置 server 的定时器。

需要注意以下要点：

- **重点**：follower 的计时器时间要在某个范围内具有随机性，总之要避免它们同时触发选举。

2.10 Make

函数原型：`func Make(peers []*labrpc.ClientEnd, me int, persister *Persister, applyCh chan ApplyMsg) *Raft`

用于创建一个 server，并初始化该 server 的某些参数。

需要注意以下要点：

- 每个 server 的初始状态都是 follower
- 在创建时就要重置 server 的定时器，否则没有 server 会触发 election。
- 在这里可以不管 `matchIndex` 和 `nextIndex` 的初始值，因为每次当一个 server 成为 leader 时都会重置它们。

2.11 Start

函数原型：`func (rf *Raft) Start(command interface{}) (int, int, bool)`

用于让 cilient 向 leader 提交一个 command。

需要注意以下要点：

- 不是 leader 的 server 收到 command 提交请求，一律拒绝。
- 只有 leader 才能接受 cilient 的请求。
- 虽然有些自以为是 leader 但实际上不是 leader 的 server 会接受 cilient 的 command，但是这些

command 都不会被 commit，因为 command 被 commit 的前提是要有超过半数的 server 接受 entry 的复制。而这超过半数的 server 中，一定有人知道这个假 leader 实际上不是 leader。

2.12 实验结果

```
PS C:\Users\dengxiao\go\src\raft> go test -run Election
Test: initial election ...
... Passed
Test: election after network failure ...
... Passed
PASS
ok      raft      7.233s
PS C:\Users\dengxiao\go\src\raft> go test -run FailNoAgree
Test: no agreement if too many followers fail ...
... Passed
PASS
ok      raft      3.643s
PS C:\Users\dengxiao\go\src\raft> go test -run ConcurrentStarts
Test: concurrent Start()s ...
... Passed
PASS
ok      raft      0.713s
PS C:\Users\dengxiao\go\src\raft> go test -run Rejoin
Test: rejoin of partitioned leader ...
... Passed
PASS
ok      raft      3.999s
PS C:\Users\dengxiao\go\src\raft> go test -run Backup
Test: leader backs up quickly over incorrect follower logs ...
... Passed
PASS
ok      raft      15.517s
PS C:\Users\dengxiao\go\src\raft> |
```

图 4 Part II 实验结果

3 Fault tolerant (Part III)

3.1 变量分析

Raft 中有 3 个非易失变量：currentTerm、votedFor、log[]，要把这三个变量存储于非易失存储设备上。`persist()`函数用于将这几个变量存到非易失性设备上，而 `readPersist()`则用于从非易失性存储设备上读出这些数据。

➤ 为什么 currentTerm 是非易失？

- | | | | |
|---|---|---|---|
| A | 1 | 1 | 2 |
| B | 1 | 1 | |
| C | 1 | 1 | |

如上，A、B、C 代表 3 个 server，这些数字是 leader 的 term。假设 A 先是 leader，在第一次任期时给 B、C 复制了两条 term=1 的 log entry。随后所有 A、B、C 都故障，都恢复后 A 又赢得选举，成为第 2 任 leader。但很不幸，A 在第二任期内还没有向其他 server 复制任何一条 log entry，所有 server 就又宕机了。更不幸的是，之后 A 启动不了了，只有 B 和 C 能正常启动，如果不把 currentTerm 记录在非易失性存储，B、C 根据 log 的内容都会认为下一任 leader 的 term 是 2，但实际上应该是 3。所以必须把 currentTerm 放在非易失存储上，这样在整个系统崩

溃后 B、C 都知道，下一任 leader 的 term 至少应该是 3。否则当 A 又可以启动时，系统中会存在两个 “term=2” 的 leader。

- 为什么 votedFor 是非易失？
 - 防止在一段时间内同时给两个 server 投票。假设 A、B 几乎同时发起选举，而给 A 投票之后马上故障了。很快恢复过来后又收到 B 的投票邀请，如果不记录之前给谁投票，那么此时又会给 B 投票，这是不允许的。
- 为什么 logs[] 是非易失？
 - 虽然 leader 一定记录着所有有效的 Entry，但有可能由于自然灾害，所有 server 同时关机了，如果不把 logs 记录在非易失性存储上，则 state machine 的状态将永远恢复不过来。
 - logs 一般是很多的，不能每次重启后都依赖网络从 leader 那里拿到所有 logs，因为那样开销太大了。
- 为什么 commitIndex、lastApplied 可以存储在易失存储上？
 - 因为从 log 中可以判断哪些 log entry 已经被过半数 server 存储，进而找到 commitIndex。
 - 如果没有检查点，server 每次重启都要从第 0 个 command 开始重新执行。如果有检查点，检查点会告诉你 lastApplied。
- 为什么 nextIndex[]、matchIndex[] 可以存储在易失存储上？
 - 每次刚成为 leader 都要重新初始化这两个数组，每个 nextIndex 的元素都初始化为 leader 的 log 的长度+1；每个 nextIndex 的元素都初始化为-1（有效下标从 0 开始），如果有效下标从 1 开始，则初始化为 0。
- 为什么 server 的状态 state 可以存储在易失存储上？
 - 因为有“超时”机制，就算存了，下次也用不上。
- 还有一些变量存了下次重启之后也用不上的，就不列举了。

综上分析，一个 server 只有 currentTerm、votedFor、log[] 这 3 个变量需要存储于非易失性设备上。那么每次修改完这些值之后，**马上**调用 **persist** 函数把它们存入非易失存储设备即可。同时为了避免线程竞争导致数据不确定，在每个需要**写或读后写** server 的成员变量时都要**先拿锁**才能写，或读后写。在 server 初始化时不需要拿锁，因为刚初始化时 server 还没有多线程并发。每次故障重启后，用 **readPersist** 重新读出 currentTerm、votedFor、log[] 即可。

实现时需要注意：

- 拿锁后记得手动放锁，想放锁得先拿锁。
- 想偷懒，也可以用 “rf.mu.Lock() defer rf.mu.Unlock()” 在进入函数时自动拿锁，返回前自动放锁。
- 当前代码在 ReliableChurn 测试上，有时通不过。

3.2 实验结果

```
PS C:\Users\dengxiao\go\src\raft> go test -run Persist1
Test: basic persistence ...
... Passed
PASS
ok      raft      3.423s
PS C:\Users\dengxiao\go\src\raft> go test -run Persist2
Test: more persistence ...
... Passed
PASS
ok      raft      15.611s
PS C:\Users\dengxiao\go\src\raft> go test -run Persist3
Test: partitioned leader and one follower crash, leader restarts ...
... Passed
PASS
ok      raft      1.650s
PS C:\Users\dengxiao\go\src\raft> go test -run Figure8
Test: Figure 8 ...
... Passed
Test: Figure 8 (unreliable) ...
... Passed
PASS
ok      raft      74.495s
PS C:\Users\dengxiao\go\src\raft> go test -run UnreliableAgree
Test: unreliable agreement ...
... Passed
PASS
ok      raft      1.971s
PS C:\Users\dengxiao\go\src\raft> go test -run ReliableChurn
Test: churn ...
... Passed
PASS
ok      raft      16.383s
PS C:\Users\dengxiao\go\src\raft> go test -run UnreliableChurn
Test: unreliable churn ...
... Passed
PASS
ok      raft      16.331s
PS C:\Users\dengxiao\go\src\raft> |
```

图 5 Part III实验结果

4 总结

纸上得来终觉浅，绝知此事要躬行。

了解理论只是第一步，只有自己去敲代码才更能体会到其中的难点和细节。感谢测试用例，否则我永远不知道错在哪里。

写此总结，望以后除了上帝，我也还能看懂代码。

References

- [1] Diego Ongaro and John Ousterhout. In Search of an Understandable Consensus Algorithm. USENIX , ATC, 2014
- [2] MIT 6.824 2020 course. <https://www.bilibili.com/video/BV1CU4y1P7PE>
- [3] MIT 6.824 Lab guidance. <https://pdos.csail.mit.edu/6.824/labs/guidance.html>